SkillLearn: Learning and Placement Preparation Platform

A PROJECT REPORT

Submitted in partial fulfilment

of the

requirement for the award of the degree of

BACHELOR OF TECHNOLOGY (B. Tech)

in

Computer Science and Engineering

by

Ravi Ranjan

229301653

Under the supervision of

Dr. Dibakar Sinha

MANIPAL UNIVERSITY JAIPUR

(University under Section 2(f) of the UGC Act)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING,
SCHOOL OF CSE,
MANIPAL UNIVERSITY JAIPUR,
RAJASTHAN, INDIA-303007

MAY, 2026

Department of Computer Science and Engineering
School of Computer Science and Engineering, Manipal University Jaipur,
Dehmi Kalan, Jaipur, Rajasthan, India- 303007

STUDENT DECLARATION

I hereby declare that this project (**Title of Project**) is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the University or other Institute, except where due acknowledgements has been made in the text.

Place: **Jaipur**
Date:11 May 2026

Ravi Ranjan
229301653
B.Tech (CSE) 8th Semester

Date: 11th May, 2026

CERTIFICATE

This is to certify that the project entitled "SkillLearn: Learning and Placement Preparation Platform" is a Bonafide work carried out as part of the course CS 4270, under my guidance from 5 Jan 2026 to 10 May 2026 by Ravi Ranjan, student of Computer Science and Engineering, 8th Semester at the Department of Computer Science and Engineering, Manipal University Jaipur, during the academic semester 8th in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, at MUJ, Jaipur.

Dr. Dibakar Sinha
Project Guide, Dept of Computer Science and Engineering
Manipal University Jaipur

Dr. Neha Chaudhary
HOD, Dept of Computer Science and Engineering
Manipal University Jaipur

1. INTRODUCTION

1.1 Introduction to the Work

The transition from college to corporate life is one of the most challenging phases in an engineering student's academic journey. Campus placement drives are structured, time-bound, and highly competitive, requiring students to demonstrate competence across multiple domains — quantitative aptitude, computer science fundamentals, data structures and algorithms, and communication skills. While there is no shortage of preparation resources on the internet, the sheer volume and fragmented nature of available content often overwhelm students and prevent them from following a structured, measurable learning path.

The Learning and Placement Preparation Platform was conceived as a direct response to this problem. Rather than offering yet another standalone quiz website or a disconnected collection of video lectures, this platform integrates four distinct but interconnected preparation tools into a single cohesive system. Students can practise computer science fundamentals through an interactive quiz engine, improve their coding skills through curated programming questions, evaluate and enhance their resumes through automated ATS-based analysis, and strengthen their domain knowledge through structured video-based learning modules — all from within the same interface, while their progress and performance are continuously tracked across sessions.

The platform was developed as a full-stack web application using MongoDB for database management, Express.js and Node.js for backend API development, and React.js with Tailwind CSS for the frontend interface. This technology combination was selected because it provides a unified JavaScript-based development environment, reducing context-switching between programming languages and enabling rapid development and scalability. MongoDB's document-oriented architecture is particularly suitable for handling flexible educational data structures such as quiz categories, coding topics, course modules, and user progress records. React.js, combined with Tailwind CSS, enables the creation of a responsive, component-based user interface that delivers a smooth experience across desktop and mobile devices.

One of the most technically significant aspects of the project is the integration of the Cohere API into the ATS Resume Checker module. Resume evaluation and ATS compatibility analysis are generally associated with enterprise-level recruitment software and paid career services. This platform brings similar functionality directly to students by allowing them to upload resumes and compare them against job descriptions. The system analyses keyword alignment, identifies missing technical skills, and generates structured feedback reports that help students improve their resumes according to industry expectations.

The application was fully deployed to the cloud to ensure accessibility and real-world usability. The backend services were hosted on Render, while the frontend application was deployed on Vercel, enabling users to access the platform from any internet-connected device. This deployment process also provided practical exposure to production-level application hosting, API integration, environment variable management, authentication handling, and cross-origin resource sharing (CORS) configuration.

Overall, this project demonstrates how modern full-stack web technologies can be combined with intelligent resume analysis and interactive learning systems to create a unified placement preparation platform. By integrating learning, assessment, progress tracking, and resume evaluation into a single environment, the platform aims to reduce the gap between academic preparation and industry readiness for engineering students.

1.2 ***Problem Statement***

Despite the rapid growth of online learning platforms globally, engineering students preparing for campus placements still face a significant structural challenge: the resources required for complete placement preparation are distributed across multiple disconnected platforms. A student may use one website for cs fundamental, another platform for coding problems, a separate paid service for video courses, and external tools or manual reviews for resume evaluation. This fragmented approach creates several practical difficulties and prevents students from following a structured and measurable preparation strategy.

One major issue is the lack of consolidated progress tracking. Since preparation activities are spread across different platforms, students are unable to obtain a unified overview of their performance and improvement areas. For example, a student may perform well in aptitude tests but remain unaware of weaknesses in coding fundamentals or technical concepts due to the absence of integrated analytics. In addition, maintaining multiple accounts and subscriptions increases both complexity and financial burden for students.

Another critical problem is the limited accessibility of professional resume feedback. Resume quality plays a vital role in determining whether a candidate successfully clears the initial screening stages of placement drives. However, many resume evaluation tools are either costly, inaccessible to students, or designed primarily for corporate recruitment environments rather than educational use cases. As a result, students often submit resumes without understanding whether they align with industry expectations or applicant tracking systems (ATS).

The absence of a unified and affordable platform that combines aptitude preparation, coding practice, structured learning, analytics, and resume evaluation highlights a genuine gap in the current

educational technology ecosystem, especially for engineering students in Indian universities. This project aims to bridge that gap by providing a single full-stack web platform where students can prepare for placements systematically, monitor their performance, improve their resumes, and access learning resources through one integrated system.

1.2.1 ***Objectives of the Project***

The primary objectives of this project are as follows:

- To design and implement a unified web platform that consolidates CS fundamentals practice, coding preparation, resume analysis, and course learning into a single product.
- To build a quiz engine with multiple categories, automatic scoring, and visual analytics so that students can measure their performance over time.
- To develop a coding preparation module with difficulty-based and topic-based filtering to support systematic practice.
- To integrate the Cohere AI API to perform intelligent, NLP-based analysis of student resumes against job descriptions, generating ATS compatibility scores and improvement suggestions.
- To create a course platform with payment gateway integration, video streaming support, and module-level progress tracking.
- To implement a secure, token-based authentication system using JSON Web Tokens (JWT) to protect user data and ensure that only authorised users can access platform features.
- To deploy the complete application on cloud infrastructure (Render + Vercel) and make it publicly accessible.

1.3 ***Scope of the Project***

The scope of this project covers the complete design, development, and deployment of a full-stack web application. The platform is intended for use by undergraduate engineering students, primarily those from Computer Science and related branches, who are preparing for technical campus placement interviews.

Within the scope of this project, the following deliverables are included: a working React.js frontend with a responsive Tailwind CSS design, a Node.js and Express.js backend with RESTful APIs, a MongoDB database with well-structured schemas for all modules, JWT-based authentication for all protected routes, an ATS resume analysis engine powered by the Cohere API, a payment-gated

course enrollment system, quiz analytics with Chart.js visualisations, and cloud deployment on Render and Vercel.

The following aspects are outside the current scope of the project: a dedicated mobile application (though the web frontend is mobile-responsive), real-time collaborative features, integration with university placement management systems, automated job board listing, and administrative content management tools beyond basic data insertion.

2. REQUIREMENT ANALYSIS

Requirement analysis is a fundamental phase in the software development lifecycle because it establishes the conceptual and technical foundation for the entire system. This phase focuses on identifying user expectations, defining operational requirements, understanding system behavior, and determining the technological and architectural needs necessary for successful implementation. A well-structured requirement analysis process significantly reduces development complexity, minimizes implementation errors, improves project planning, and ensures that the final software product fulfills practical user requirements effectively.

The Learning and Placement Preparation Platform was conceptualized and developed to provide students with a centralized environment for placement-oriented preparation and learning management. In conventional placement preparation workflows, students generally depend on multiple disconnected platforms for aptitude preparation, coding practice, resume optimization, technical learning, and performance tracking. Such fragmentation often creates difficulties in maintaining consistency, tracking progress, and organizing preparation activities efficiently. The proposed platform addresses these challenges by integrating all major preparation modules into a single full-stack web application developed using the MERN stack architecture.

The platform incorporates several important functionalities, including aptitude and technical quizzes, coding preparation modules, ATS-based resume analysis, course management systems, learning analytics dashboards, and secure user authentication mechanisms. During the requirement analysis phase, considerable attention was given to understanding the practical challenges faced by students during placement preparation and identifying the features necessary to improve learning efficiency, preparation quality, and user experience.

The requirement analysis process also played an important role in defining the system scope, frontend-backend communication mechanisms, database architecture, security requirements, deployment strategies, and overall application workflow. Both functional and non-functional requirements were analyzed carefully to ensure that the developed platform remains scalable, maintainable, responsive, secure, and suitable for future expansion.

2.1 ***Functional Requirements***

Functional requirements describe the operations and services that the system must perform in order to satisfy user expectations and achieve the intended objectives of the application. These requirements define how different modules interact with users and explain the behavior of the application under different operational scenarios. Functional requirements directly influence the design of frontend interfaces, backend APIs, database structures, and overall system workflows.

The Learning and Placement Preparation Platform consists of several interconnected modules, each designed to support a specific aspect of placement preparation and learning management. The major functional requirements identified during system analysis are discussed below.

2.1.1 User Registration and Authentication System

The platform must provide a secure and efficient user registration and authentication mechanism to ensure controlled access to system functionalities. New users should be able to create accounts by providing basic details such as name, email address, and password. During the registration process, the system must validate all user inputs and ensure that duplicate accounts are not created using the same email address.

The backend system should securely encrypt user passwords before storing them in the MongoDB database. Password hashing is necessary to protect sensitive user credentials and improve overall application security. After successful registration, users should be redirected to the login interface where they can authenticate themselves using valid credentials.

The authentication module must generate JWT-based authentication tokens after successful login verification. These tokens should be used to manage secure sessions and protect restricted resources within the application. Only authenticated users should be allowed to access protected modules such as quizzes, coding preparation dashboards, ATS resume analysis systems, analytics dashboards, and enrolled learning courses.

The authentication workflow should also support secure logout functionality to terminate user sessions properly. Middleware-based authorization mechanisms must validate authentication tokens before allowing access to protected backend APIs and frontend routes.

2.1.2 Quiz Management System

The quiz management system represents one of the primary functionalities of the platform because it supports aptitude preparation and technical interview preparation for students. The system must provide a structured environment where users can attempt multiple-choice quizzes related to aptitude and computer science subjects.

The quiz module should support multiple subject categories such as Operating Systems, Database Management Systems, Computer Networks, Data Structures, Object-Oriented Programming, Aptitude, and Logical Reasoning. Users should be able to select categories according to their preparation requirements and attempt quizzes dynamically retrieved from the database.

The frontend should display questions along with multiple answer options in an organized manner. Users must be able to select answers and submit quizzes after completion. Once the quiz is

submitted, the backend should evaluate user responses automatically by comparing selected answers with correct answers stored inside MongoDB collections.

After evaluation, the system should generate detailed result summaries displaying the total number of questions attempted, correct answers, incorrect answers, and percentage scores. The platform should also maintain historical quiz records to allow students to monitor improvement over time and analyze performance consistency across different subjects.

2.1.3 ATS Resume Analysis System

The ATS Resume Analysis module is one of the most important components of the platform because it integrates resume evaluation and placement-oriented analysis into the learning environment. Modern recruitment processes often rely on Applicant Tracking Systems (ATS) to filter resumes before they are reviewed by recruiters. Therefore, students must optimize resumes according to ATS standards.

The platform should allow users to upload resumes and compare them against job descriptions for compatibility analysis. The workflow begins when the user uploads a resume and provides a corresponding job description. The backend system extracts textual content from the uploaded resume and combines it with job description data before generating prompts dynamically for AI-based analysis systems.

The ATS analysis process should generate ATS scores, identify missing technical skills, analyze keywords, and provide resume improvement suggestions. These functionalities help students understand weaknesses in resumes and improve resume quality according to industry expectations and recruiter standards.

2.1.4 Analytics Dashboard System

The analytics dashboard is responsible for visualizing user performance and learning progress through graphical representations. Performance analysis is extremely important during placement preparation because students need continuous feedback regarding strengths, weaknesses, and preparation consistency.

The analytics module should retrieve historical quiz data and learning records from the database and generate interactive visualizations such as bar charts, score comparison graphs, and historical tracking reports. These graphical representations help students analyze preparation trends and identify areas requiring improvement.

The analytics system should support responsive rendering so that dashboards remain accessible across desktop systems, tablets, and mobile devices.

2.1.5 Cloud Deployment Support

The platform must support deployment on cloud hosting platforms to ensure production readiness and accessibility. The frontend application should support deployment on Vercel, while backend APIs should support deployment on Render.

MongoDB Atlas should be used for cloud database hosting. Environment variables must be configured securely to protect database credentials, authentication secrets, and API keys. Cloud deployment architecture improves scalability, accessibility, reliability, and production-level hosting support.

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational standards of the platform. Unlike functional requirements, which describe system operations, non-functional requirements explain how efficiently and reliably the application performs under different conditions.

2.2.1 Performance Requirements

The platform should provide smooth user interaction and fast response times to ensure a positive user experience. Educational and placement preparation systems require efficient frontend rendering and optimized backend processing because delays in quizzes, analytics dashboards, and ATS analysis workflows negatively affect usability.

The frontend should load pages efficiently and minimize unnecessary rendering operations. Backend APIs should process requests quickly while reducing unnecessary database operations. MongoDB queries should also be optimized to improve data retrieval speed and overall system responsiveness.

2.2.2 Maintainability Requirements

The project architecture should remain organized to simplify debugging, future development, and feature enhancement. The backend structure separates controllers, routes, middleware, models, and configuration files into independent directories.

Similarly, the frontend organizes components, pages, services, and utility functions separately to improve code readability and maintainability.

2.2.3 Reliability Requirements

The platform must operate consistently under different operational conditions without frequent failures or data inconsistencies. Backend APIs should handle invalid requests gracefully and maintain stable communication with the database.

The application should support consistent deployment workflows, stable API communication, and reliable database operations. MongoDB Atlas cloud hosting improves reliability through managed services and backup support.

2.3 Use Case Scenarios

Use case scenarios describe the interaction between users and the system under practical operational conditions. These scenarios provide a clear understanding of how different modules of the platform behave when users perform specific activities. Use case analysis is extremely important during software development because it helps developers understand workflow sequences, frontend-backend communication, database interactions, and user-system relationships.

In the Learning and Placement Preparation Platform, use case scenarios were designed to represent real-world student activities such as registration, login, quiz participation, coding preparation, ATS resume analysis, course enrollment, and analytics monitoring. These workflows ensure that all functionalities operate in a structured and user-friendly manner.

The following subsections explain the major use case scenarios implemented within the platform.

2.3.1 User Registration Scenario

The user registration process represents the initial interaction between the user and the platform. The primary objective of this workflow is to create a secure and authenticated user profile that enables access to platform functionalities. Since the application stores user-specific preparation data, analytics records, ATS reports, and course enrollments, secure registration and identity verification are essential.

The registration workflow begins when a new user accesses the registration page through the frontend interface. The system displays a registration form requesting details such as the user's full name, email address, and password. After the user submits the form, the frontend sends the entered data to backend APIs through secure HTTP requests.

The backend validates all input fields carefully to ensure data consistency and prevent invalid entries. The validation process checks whether required fields are empty, whether the email format is correct, and whether the password satisfies minimum security conditions. The backend also checks the MongoDB database to verify whether another account already exists using the same email address.

If validation succeeds, the password is encrypted using hashing techniques before storage. Password hashing improves application security because original passwords are never stored directly inside the database. After successful account creation, the system stores user information inside MongoDB collections and redirects the user to the login interface.

This registration workflow ensures:

- Secure user onboarding

- Proper data validation
- Prevention of duplicate accounts
- Protected credential storage

2.3.2 User Login Scenario

The login workflow allows registered users to access protected functionalities within the platform. Since multiple modules such as quizzes, ATS analysis systems, analytics dashboards, and course management systems contain personalized data, authentication becomes necessary for maintaining security and controlled access.

The login process begins when the user enters valid credentials through the login interface. The frontend transfers authentication details securely to backend APIs. The backend then searches MongoDB collections for matching user records.

Once the user record is located, the backend compares the entered password with the hashed password stored in the database. If authentication succeeds, the system generates a JWT (JSON Web Token). This token acts as a secure session identifier and is used to authorize future API requests.

The generated token is stored temporarily within browser storage mechanisms and attached automatically with future backend requests. Protected routes within the application validate JWT tokens before allowing access to secure resources.

If authentication fails due to incorrect credentials, the system generates appropriate error messages and denies access.

The login workflow improves:

- Secure session management
- User-specific dashboard access
- API-level protection
- Controlled authorization

2.3.3 Quiz Attempt Scenario

The quiz participation workflow was designed to support aptitude and technical interview preparation through structured assessment systems. This scenario explains how students interact with the quiz module to evaluate preparation progress and improve subject understanding.

The workflow begins when an authenticated user accesses the quiz dashboard and selects a preferred subject category such as Operating Systems, DBMS, Computer Networks, Aptitude, or Data Structures. The frontend sends requests to backend APIs to retrieve questions associated with the selected category.

The backend communicates with MongoDB collections and returns structured quiz data to the frontend interface. The frontend dynamically renders questions along with multiple answer options in an organized format.

During quiz participation, users can navigate through questions, select answers, and submit responses after completion. Once the quiz is submitted, backend APIs compare user responses with correct answers stored inside MongoDB collections.

The backend then calculates:

- Total score
- Correct answers
- Incorrect answers
- Percentage performance

2.3.4 ATS Resume Analysis Scenario

The ATS Resume Analysis workflow is one of the most important scenarios in the platform because it integrates resume optimization and placement-oriented analysis functionality into the system.

The process begins when a user uploads a resume file and enters a job description through the ATS analysis interface. The frontend transfers uploaded documents and job description content to backend APIs securely.

The backend extracts textual content from the uploaded resume using parsing techniques. After extracting resume data, the backend combines it with the provided job description and generates prompts dynamically for AI-based analysis systems.

The AI analysis system evaluates:

- Resume relevance
- Keyword matching
- Missing technical skills
- ATS compatibility
- Resume optimization suggestions

The generated response contains ATS scores and detailed improvement recommendations. The backend processes this response and stores analysis reports inside MongoDB collections for future access.

Finally, the frontend displays the generated ATS report in a structured and user-friendly dashboard. Students can analyze weaknesses in resumes and improve resume quality according to industry requirements.

This workflow significantly improves:

- Resume optimization
- ATS compatibility understanding
- Placement readiness
- Technical profile enhancement

2.3.5 Course Enrollment Scenario

The course enrollment workflow supports online learning functionalities within the platform. This module was designed to provide students with structured educational content integrated directly with placement preparation activities.

The workflow begins when users browse available courses through the learning dashboard. Users can search for courses using keywords and filter learning modules according to categories or technical domains.

After selecting a preferred course, the frontend sends enrollment requests to backend APIs. The backend verifies user authentication and creates enrollment records inside MongoDB collections.

Once enrollment is completed, users gain access to course content, video lessons, and learning modules. The system dynamically tracks lesson completion status and updates progress percentages whenever users complete learning activities.

2.3.6 Analytics Dashboard Scenario

The analytics dashboard workflow focuses on performance visualization and preparation monitoring. Continuous performance evaluation is extremely important during placement preparation because students need feedback regarding learning progress and preparation consistency.

The analytics workflow begins when users access the analytics dashboard after completing quizzes or learning activities. The frontend sends requests to backend APIs for retrieving historical performance data.

The backend retrieves quiz scores, preparation records, and learning progress information from MongoDB collections. This data is processed and converted into graphical representations using chart libraries such as Recharts and Chart.js.

The dashboard displays:

- Bar charts
- Historical performance trends

- Progress tracking reports

2.4 Other Software Engineering Methodologies

Software engineering methodologies play an important role in the successful development of large-scale web applications because they provide structured approaches for designing, implementing, testing, deploying, and maintaining software systems. The selection of suitable methodologies directly affects scalability, maintainability, performance, development speed, and overall software quality.

During the development of the Learning and Placement Preparation Platform, several modern software engineering methodologies and architectural principles were adopted to ensure that the system remains organized, scalable, secure, and suitable for future enhancements. Since the platform combines multiple modules such as quizzes, coding preparation systems, ATS resume analysis, analytics dashboards, and course management systems, the use of structured development methodologies was necessary to simplify implementation and improve maintainability.

The following subsections explain the major software engineering methodologies and architectural approaches followed during the development of the project.

2.4.1 Modular Development Methodology

The platform follows modular development principles where each module was implemented independently before integration into the complete system. Authentication systems, quizzes, coding preparation modules, ATS analysis systems, analytics dashboards, and course management systems were developed separately to simplify debugging and feature expansion.

2.4.2 MVC Architecture

The backend architecture of the platform follows the Model-View-Controller (MVC) design pattern. MVC is one of the most widely used architectural methodologies in full-stack web development because it separates application logic into independent components.

The MVC architecture divides the system into three primary layers:

- Model Layer
- View Layer
- Controller Layer

The model layer is responsible for managing database schemas and MongoDB collections. It defines the structure of application data such as users, quizzes, ATS reports, enrollments, and coding questions.

The controller layer contains business logic and backend processing functionalities. Controllers manage authentication workflows, quiz evaluation systems, ATS analysis handling, progress tracking, and API response generation.

The view layer represents frontend interfaces developed using React.js. These interfaces display dashboards, forms, quizzes, analytics graphs, ATS reports, and learning modules to users.

The implementation of MVC architecture improves:

- Separation of concerns
- Code maintainability
- Project organization
- Scalability
- Readability
- Backend management efficiency

The MVC pattern also simplifies future modifications because frontend interfaces, backend logic, and database structures operate independently while maintaining communication through APIs.

3. BACKGROUND MATERIAL

3.1 ***Review of Existing Systems***

Several platforms currently exist in the edtech and career preparation space, each addressing a subset of the problem this project targets. Understanding their strengths and limitations helped shape the design decisions made in this project.

3.1.1 ***LeetCode and HackerRank***

LeetCode and HackerRank are perhaps the most widely used platforms for coding practice among engineering students. Both offer extensive libraries of programming problems categorised by difficulty and topic, with online judges that evaluate submitted code automatically. However, these platforms are narrowly focused on algorithmic problem-solving and do not provide aptitude preparation, resume analysis, or structured learning content. A student using LeetCode exclusively must look elsewhere for all other aspects of placement preparation.

3.1.2 ***Resumeworded and Jobscan***

Resumeworded and Jobscan are AI-powered resume analysis tools that offer ATS scoring and keyword analysis. They are effective but are not oriented toward students — they are commercial tools designed for job-seeking professionals and come with subscription fees. They do not integrate with any learning or practice content, meaning a student using these tools still needs to go elsewhere for their preparation needs.

3.1.3 ***Coursera, Udemy, and NPTEL***

Platforms like Coursera, Udemy, and NPTEL offer high-quality video-based course content, often taught by industry professionals or university professors. They address the learning content gap but are not designed for placement preparation in a structured sense. There is no aptitude quiz system, no coding practice environment, and no resume analysis capability. Payment models on Coursera and Udemy can also be prohibitively expensive for many students.

3.2 ***Gap Analysis***

From this review, a clear gap emerges: there is no single platform that combines aptitude quizzes, coding practice, AI-powered resume analysis, and paid course learning in one unified product,

particularly one that is designed with Indian engineering students and campus placement preparation in mind. This gap is precisely what the current project addresses.

3.3 Technologies Involved

3.3.1 **MongoDB**

MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like BSON documents. Unlike relational databases, MongoDB does not require a fixed schema, making it highly adaptable for applications where the structure of data may vary across records — such as quiz questions with varying numbers of options, or course modules with different content types. In this project, MongoDB is used to store user accounts, quiz questions and results, coding problems, course data, enrollment records, and ATS report summaries. The Mongoose ODM library is used to define schemas and interact with MongoDB from within the Node.js application.

3.3.2 **Express.js**

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building REST APIs. It handles HTTP routing, middleware integration, and request/response management with a clean, middleware-based architecture. In this project, Express.js serves as the backbone of the backend, with separate route files and controller modules for each major feature area — authentication, quiz, coding, ATS, and courses.

3.3.3 **React.js**

React.js is a declarative, component-based JavaScript library for building user interfaces. Developed by Facebook and maintained as an open-source project, React allows developers to build complex UIs from small, reusable components. Its virtual DOM diffing algorithm ensures that only the parts of the actual DOM that have changed are re-rendered, resulting in fast, responsive user interfaces. In this project, React.js is used to build the entire frontend, including the quiz interface, course player, resume upload form, and analytics dashboard. React Router is used for client-side navigation between pages.

3.3.4 **Node.js**

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside the browser. It uses an event-driven, non-blocking I/O model that makes it well-suited for building scalable network applications. Node.js serves as the runtime environment for the

Express.js server in this project. Its single-threaded event loop is capable of handling a large number of concurrent connections.

3.3.5 *Tailwind CSS*

Tailwind CSS is a utility-first CSS framework that allows developers to build custom user interfaces directly in their HTML or JSX markup by composing utility classes. Unlike frameworks like Bootstrap, which provide pre-built components with opinionated styles, Tailwind provides low-level utility classes that can be combined to create virtually any design. This approach results in smaller CSS bundles (through PurgeCSS integration) and gives developers complete control over the visual design without fighting the framework's defaults. Tailwind was used throughout this project's React frontend for layout, typography, colour, spacing, and responsive design.

3.3.6 *Payment Gateway (Razorpay)*

Razorpay is a leading Indian payment gateway that supports a wide variety of payment methods including UPI, credit cards, debit cards, and net banking. It provides a developer-friendly SDK that integrates with both web and mobile applications. In this project, Razorpay is used to handle course enrollment payments, verifying transaction success through server-side order verification using Razorpay's signature validation mechanism before granting the student access to enrolled course content.

3.3.7 *JWT Authentication*

JSON Web Tokens (JWT) are an open standard (RFC 7519) for securely transmitting information between parties as a compact, URL-safe token. A JWT consists of three parts — a header, a payload, and a signature — encoded in Base64URL format and separated by dots. In this project, JWTs are generated by the server upon successful login and sent to the client, which stores the token in local storage and includes it in the Authorization header of all subsequent API requests. The server validates the token on each request using a secret key, ensuring that only authenticated users can access protected routes.

3.3.8 *Render and Vercel*

Render is a cloud platform that offers managed hosting for web services, including Node.js applications. It provides automatic deployment from GitHub repositories, managed SSL certificates, and environment variable configuration through a web console. Vercel is a frontend deployment platform optimised for React and Next.js applications. It offers automatic CI/CD from GitHub,

global CDN distribution, and instant deployments. In this project, the Express.js backend is hosted on Render and the React.js frontend is deployed on Vercel.

3.4 ***Literature Survey***

A review of relevant academic and industry literature was conducted to contextualise this project within the broader landscape of AI-assisted education and placement preparation tools.

Lykourentzou et al. (2009) demonstrated in their study "Dropout prediction in e-learning courses through the combination of machine learning techniques" that personalised feedback and progress tracking significantly reduce dropout rates in online learning environments. This finding supports the decision to include robust analytics and progress tracking in the current platform.

The work of Doyle (2014) in "Resume Tips for Computer Science Students" highlighted that the majority of resumes submitted by engineering graduates fail ATS screening due to a lack of relevant keyword optimisation. This directly motivated the inclusion of the ATS Resume Checker module in this project.

Devlin and Samarawickrema (2010) argued in "The criteria of effective teaching in a changing higher education context" that assessment with immediate feedback is one of the most powerful drivers of learning. The quiz system in this project is designed around this principle, providing instant score reports and visual performance comparisons after each quiz attempt.

Research by Hrastinski (2008) on asynchronous and synchronous e-learning demonstrated that students benefit most from a combination of structured content delivery and self-paced practice opportunities. The combination of the course learning module and the quiz and coding practice modules in this project reflects this finding.

From an industry perspective, reports from the National Association of Colleges and Employers (NACE) consistently show that employers in the technology sector prioritise demonstrated problem-solving ability, communication skills, and relevant technical skills — all areas directly addressed by the features in this platform.

4. SYSTEM DESIGN

System design is one of the most significant phases in the software development lifecycle because it transforms the requirements gathered during system analysis into a structured technical architecture. This phase focuses on defining the organization of software components, establishing communication workflows, designing database structures, planning frontend and backend interactions, and implementing methodologies that improve scalability, maintainability, security, and performance.

The Learning and Placement Preparation Platform was designed as a centralized web-based application that combines placement preparation, coding practice, resume analysis, online learning, and performance tracking into a single integrated system. The system architecture was developed using the MERN stack consisting of MongoDB, Express.js, React.js, and Node.js. The selection of this technology stack was based on its scalability, flexibility, modularity, and suitability for modern full-stack web application development.

The design process focused on creating a platform that remains responsive, secure, modular, and scalable while supporting future feature enhancements. Since the platform integrates multiple functionalities such as quizzes, coding preparation, ATS resume analysis, course management systems, and analytics dashboards, the architecture needed to ensure smooth communication between all modules without affecting system performance.

During the design phase, special attention was given to frontend responsiveness, backend API organization, database normalization, authentication workflows, cloud deployment support, and modular code organization. The system was designed in such a way that individual modules can operate independently while maintaining efficient interaction through REST APIs.

The following sections explain the design goals, overall system architecture, and detailed software engineering methodologies used during the implementation of the platform.

4.1 ***Design Goals***

Design goals define the major objectives and quality expectations considered during system development. These goals ensure that the platform remains scalable, secure, maintainable, user-friendly, and suitable for real-world deployment.

The Learning and Placement Preparation Platform was designed by considering several technical and operational goals necessary for efficient educational and placement-oriented systems.

Goal 1: Scalability

One of the primary design goals of the project was scalability. Educational platforms often experience increasing numbers of users and continuous feature expansion over time. Therefore, the architecture needed to support future enhancements without requiring major modifications to the existing codebase.

The platform was developed using modular architecture where independent modules such as authentication systems, quiz management systems, ATS resume analysis, analytics dashboards, and course management systems operate separately while maintaining communication through APIs.

MongoDB was selected because of its flexible schema structure and efficient handling of large-scale document-oriented data. REST API architecture also improves scalability because frontend and backend systems can evolve independently.

Goal 2: Security

Security was considered one of the most critical design objectives because the platform stores sensitive information such as user credentials, ATS reports, course enrollments, quiz histories, and learning progress records.

JWT-based authentication was implemented to manage secure user sessions and protect restricted functionalities. Password hashing techniques were used before storing user credentials inside MongoDB collections. Middleware authorization was integrated to validate user tokens before granting access to protected APIs and frontend routes.

Environment variables were configured to protect:

- Database credentials
- JWT secrets
- API keys
- Deployment configurations

The security architecture ensures that unauthorized users cannot access restricted resources and user data remains protected during frontend-backend communication.

Goal 3: User-Friendly Interface

Another major design goal was improving user experience and accessibility. Since the platform is designed primarily for students preparing for placements, the interface needed to remain simple, organized, responsive, and visually consistent.

React.js was used to create reusable frontend components and dynamic interfaces. Tailwind CSS was integrated to design responsive layouts that adapt efficiently across desktop systems, tablets, and mobile devices.

The frontend design focused on:

- Easy navigation
- Structured dashboards
- Responsive layouts
- Interactive visualizations
- Clear content organization

The system ensures that users can navigate between quizzes, coding preparation modules, ATS analysis dashboards, analytics systems, and course learning sections without confusion.

4.2 System Architecture

System architecture defines the structural organization of the software application and explains how different components interact with one another to perform complete system operations. The architecture of the Learning and Placement Preparation Platform was designed carefully to ensure smooth communication between frontend interfaces, backend services, database systems, authentication mechanisms, analytics modules, and ATS resume analysis functionalities. Since the platform combines multiple modules such as quizzes, coding preparation systems, ATS analysis, learning dashboards, and course management functionalities, the architecture needed to support efficient integration without affecting system performance or scalability.

The application follows a client-server architecture in which the frontend communicates with backend APIs through HTTP requests while the backend interacts with MongoDB collections for data storage and retrieval. The complete architecture was designed to maintain modularity, scalability, maintainability, responsiveness, and production readiness. Each component of the architecture performs independent responsibilities while contributing toward the overall functionality of the platform.

The system architecture can broadly be divided into the frontend layer, backend layer, database layer, authentication layer, ATS processing layer, analytics layer, and deployment layer. These layers communicate with each other in a structured manner to ensure smooth workflow execution throughout the application.

4.2.1 Overall System Architecture

The overall system architecture represents the complete interaction workflow between users, frontend interfaces, backend services, database systems, external AI services, and cloud deployment platforms. The architecture was designed in a layered format because layered systems improve maintainability and simplify debugging processes. Each layer performs specialized tasks independently while maintaining communication with other layers through structured APIs.

The workflow begins when users interact with the frontend application through desktop systems, laptops, tablets, or mobile devices. The React.js frontend receives user inputs and sends requests to backend APIs developed using Node.js and Express.js. The backend processes these requests, validates user authentication, interacts with MongoDB collections, communicates with external ATS analysis services when required, and returns structured JSON responses back to the frontend.

The frontend then dynamically renders quiz dashboards, analytics reports, ATS analysis results, coding preparation modules, and learning interfaces according to the received responses. The architecture ensures efficient communication between all modules while maintaining security, scalability, and responsiveness.

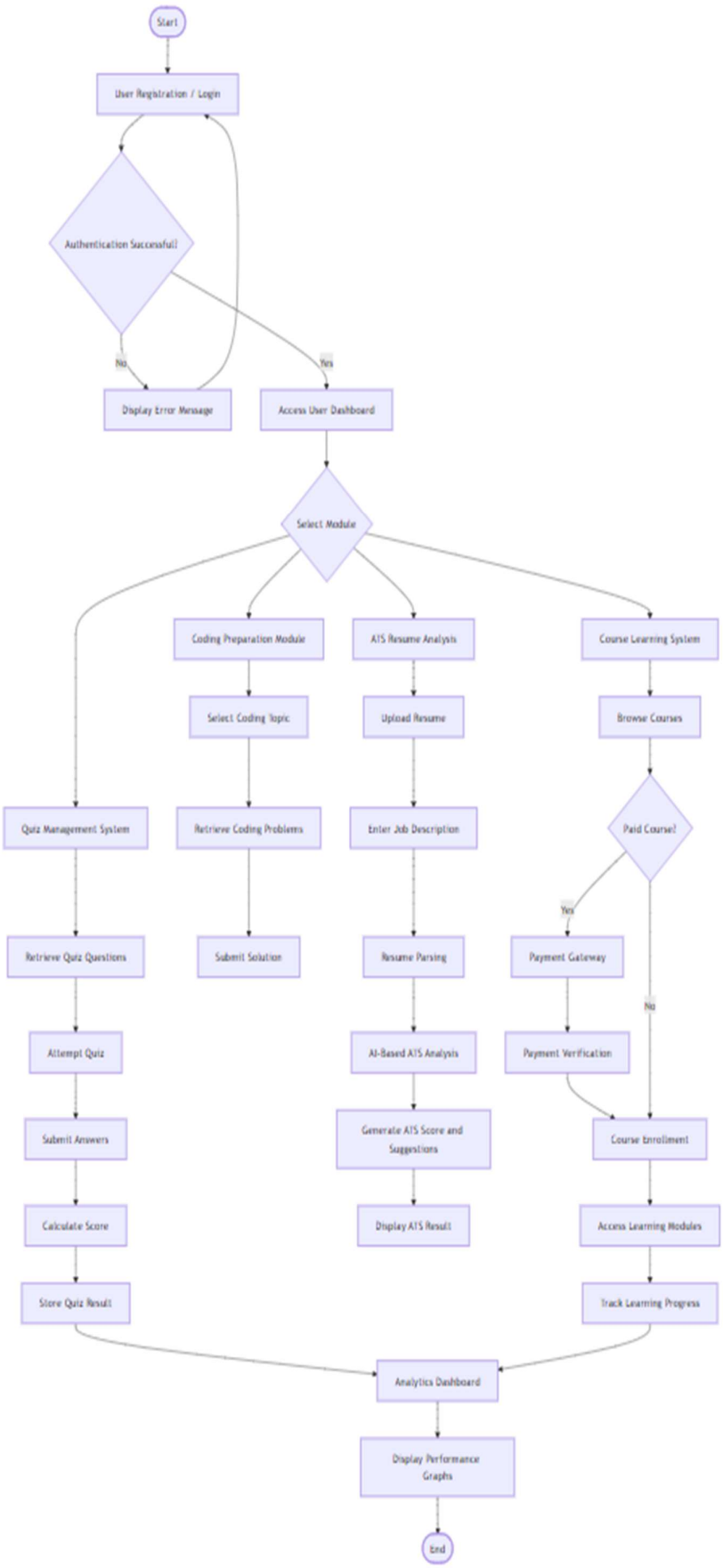


Figure 4.1 Flow Chart

4.2.2 Frontend Architecture

The frontend architecture was implemented using React.js because React supports component-based development, reusable interfaces, efficient rendering, and better state management. The frontend layer acts as the primary interaction point between users and the system. Every operation performed by users begins through frontend interfaces and is processed dynamically using React components.

The frontend contains several interfaces including authentication pages, quiz dashboards, coding preparation systems, ATS analysis modules, course learning dashboards, and analytics visualization panels. Instead of designing each interface independently from scratch, reusable React components were created to improve consistency and simplify development workflows. Components such as navigation bars, forms, dashboard cards, charts, buttons, and learning panels are reused across different sections of the platform.

Whenever users perform actions such as logging into the system, attempting quizzes, uploading resumes, or enrolling in courses, the frontend sends requests to backend APIs using HTTP communication methods. Based on backend responses, React dynamically updates the user interface without requiring complete page reloads. This significantly improves user experience and application responsiveness.

Tailwind CSS was integrated extensively throughout frontend development to create responsive and visually organized layouts. The interface dynamically adjusts according to screen size and device resolution, ensuring accessibility across mobile devices, tablets, and desktop systems.

The frontend architecture improves interface responsiveness, navigation simplicity, reusable component development, and overall user experience. It also simplifies future frontend expansion because new components can easily be integrated into the existing architecture.

4.2.3 Backend Architecture

The backend architecture was developed using Node.js and Express.js. The backend acts as the central processing unit of the platform because it manages authentication workflows, business logic execution, database communication, ATS processing, analytics generation, and API response handling.

The backend follows REST API architecture where different endpoints manage different functionalities independently. Express.js was selected because it simplifies API creation and middleware integration while supporting scalable server-side development.

Whenever frontend interfaces send requests, backend APIs process those requests and determine which operation should be executed. For example, when users attempt quizzes, backend APIs

retrieve quiz questions from MongoDB collections, process answer submissions, calculate scores, and store results for future analytics generation.

Similarly, when users upload resumes for ATS analysis, backend APIs extract textual data from uploaded documents, generate prompts dynamically, communicate with AI analysis services, and process generated ATS reports before storing them inside the database.

The backend structure separates controllers, routes, middleware, models, and configuration files into independent directories. Controllers manage operational logic while routes define API endpoints. Middleware handles authentication validation and request verification before allowing access to protected resources.

This architecture significantly improves scalability, maintainability, debugging efficiency, and backend organization. Since all functionalities are modularized properly, future features can be integrated into the backend without affecting existing workflows.

4.2.4 ***Database Architecture***

MongoDB was selected as the primary database system because of its flexible document-oriented architecture and efficient handling of dynamic application data. Since the platform manages multiple types of information such as quiz records, ATS reports, user authentication data, learning progress, and course enrollments, a flexible NoSQL database structure was more suitable than traditional relational database systems.

The database architecture was designed carefully to maintain separation between different application modules. Independent collections were created for user information, quizzes, ATS reports, course enrollments, coding questions, and analytics records. This separation improves organization and simplifies query management.

The Users collection stores authentication credentials and profile-related information. Quiz collections contain technical and aptitude questions categorized according to subjects. Quiz Result collections store performance history and score records which are later used by analytics dashboards.

ATS Reports collections store generated ATS scores, resume analysis results, and keyword suggestions. Course collections manage learning modules while Enrollment collections maintain student progress tracking information.

Relationships between collections were maintained using ObjectId references while preserving MongoDB flexibility. The database structure supports fast document retrieval, scalable data management, and flexible schema modification.

MongoDB Atlas was selected for cloud database hosting because it provides managed cloud services, backup support, scalability, and reliable database accessibility.

4.3 ***ER Diagram and Database Design***

The database architecture of the Learning and Placement Preparation Platform was designed using MongoDB to provide scalable, flexible, and efficient management of application data. Since the platform integrates multiple modules such as user authentication, aptitude quizzes, coding preparation systems, ATS resume analysis, online learning modules, and analytics dashboards, the database structure needed to support efficient communication between collections while maintaining scalability and maintainability.

MongoDB was selected because of its document-oriented architecture and its ability to handle dynamic application data efficiently. Unlike traditional relational databases, MongoDB stores data in BSON document format, which provides greater flexibility in managing large-scale web application data. Although MongoDB is schema-less by nature, Mongoose schemas were implemented within the backend to enforce structured validation, maintain consistency, and simplify data management throughout the application.

The database architecture contains multiple collections responsible for managing users, quiz questions, quiz attempts, coding problems, course modules, learning progress, ATS reports, and future payment integration functionalities. Relationships between collections are maintained using ObjectId references instead of SQL-based foreign key constraints. This approach improves flexibility while preserving logical relationships between data entities.

The overall database structure was designed carefully to improve:

- Scalable data management
- Efficient query processing
- Future feature integration
- Cloud database deployment support

The following sections explain the Entity Relationship Diagram and the schema structure of each major collection used in the platform.

4.3.1 ***Entity Relationship Overview***

The Entity Relationship (ER) Diagram represents the logical structure of the database and illustrates how different collections interact with each other inside the platform. Even though MongoDB does not follow strict relational database principles like SQL systems, relationships between collections were established using ObjectId references through Mongoose schemas.

The Users collection acts as the central entity because most modules interact with authenticated user records. Quiz attempts, ATS reports, coding submissions, enrollments, and learning progress records

are associated with individual users. Similarly, quiz questions are connected with subjects, coding problems are connected with topics, and course modules are linked with learning courses.

W

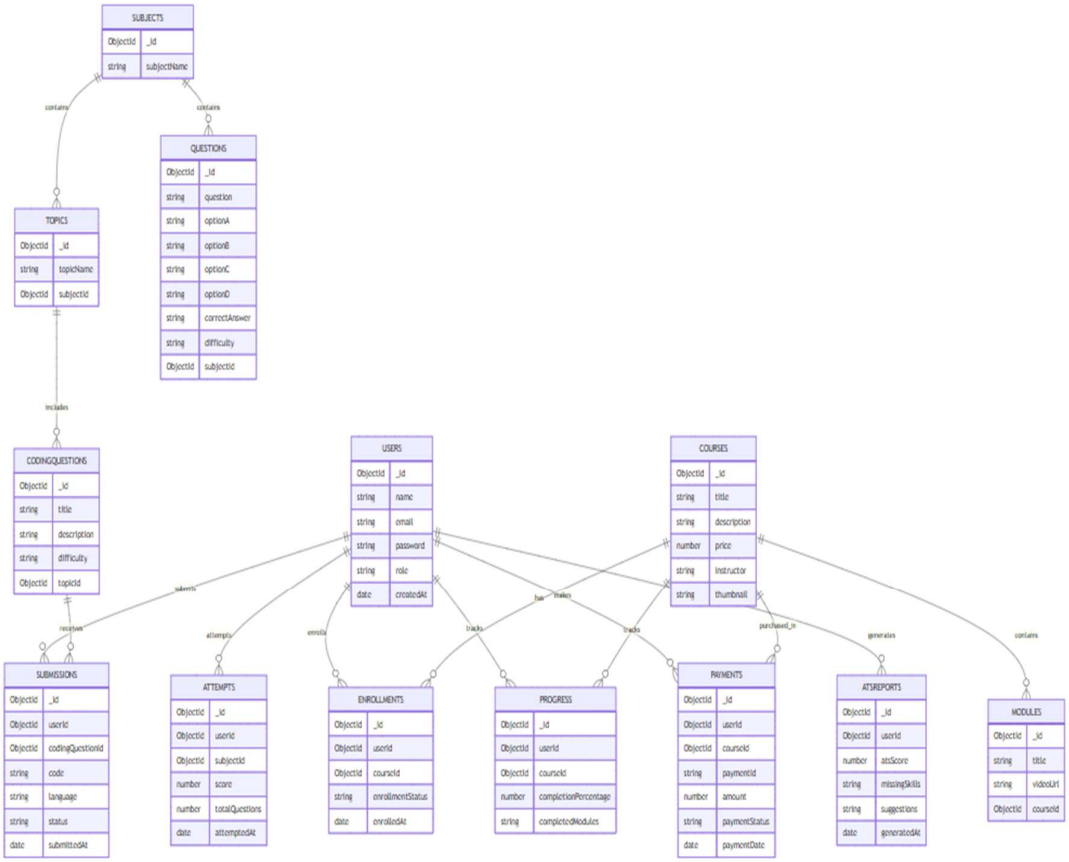


Figure 4.2 ER Diagram

4.3.2 User Schema

The User schema acts as the central component of the database architecture because all major functionalities depend upon authenticated user records. The schema stores information related to user identity, authentication credentials, and authorization roles.

The User schema was designed to support secure authentication workflows and personalized dashboard functionalities. Password hashing techniques were implemented before storing passwords inside MongoDB collections to improve security and prevent credential exposure.

The schema also supports future administrative functionalities through role-based authorization mechanisms.

Table 4.1 User Schema Description

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for each user
name	String	Stores full name of user
email	String	Stores registered email
password	String	Stores encrypted password
role	String	Defines user role

4.3.3 Subject Schema

The Subject schema stores major preparation categories used within the platform. Subjects help organize quizzes and coding preparation materials systematically.

Table 4.2 Subject Schema Description

Field Name	Data Type	Description
_id	ObjectId	Unique identifier for subject
subjectName	String	Stores subject title

4.3.4 Question Schema

The Question schema stores cs fundamental questions used within the quiz management system. Each question contains multiple answer options, correct answers, difficulty levels, and subject references.

Table 4.3 Question Schema Description

Field Name	Data Type	Description
_id	ObjectId	Unique question identifier
question	String	Stores question statement
optionA	String	First option
OptionB	String	Second optics
optionC	String	Third option
optionD	String	Fourth option

Difficulty	String	Defines difficulty level
subjectId	ObjectId	Reference to subject

4.3.5 Attempt Schema

The Attempts schema stores historical quiz records for users. Every quiz attempt generates a new attempt document containing scores, timestamps, and subject references.

Table 4.4 Attempt Schema Description

Field Name	**Data Type**	**Description**
id	ObjectId	Unique attempt identifier
userId	ObjectId	Reference to user
subjectId	ObjectId	Reference to subject
Score	Number	Stores quiz score
attemptedAt	Date	Stores attempt date

4.3.6 Coding Question Schema

The CodingQuestions schema stores coding interview problems categorized according to topics and difficulty levels.

Table 4.5 Coding Question Schema Description

Field Name	**Data Type**	**Description**
_id	ObjectId	Unique coding question ID
title	String	Coding problem title
difficulty	String	Difficulty classification
topicId	ObjectId	Reference to topic
description	String	Problem description

4.3.7 Submission Schema

The Submissions schema stores coding submissions and user practice records. This schema supports future coding evaluation systems and submission tracking functionalities.

Table 4.6 Submission Schema Description

Field Name	Data Type	Description
_id	ObjectId	Unique submission identifier
userId	ObjectId	Reference to user
codingQuestionId	ObjectId	Reference to coding question
code	String	Submitted code
status	String	Submission status

4.3.8 Course Schema

The Course schema stores online learning course information. The platform supports paid learning functionalities, therefore course records contain pricing details and instructor information.

Table 4.7 Course Schema Description

Field Name	Data Type	Description
_id	ObjectId	Unique course identifier
Title	String	Course title
Description	String	Course description
Price	Number	Course price
Instructor	String	Instructor name

4.3.9 Payment Schema

To support paid course enrollment functionalities, a Payment schema can be integrated into the platform. The payment architecture will manage transaction records, payment statuses, and course purchase histories.

Table 4.8 Payment Schema Description

Field Name	Data Type	Description
_id	ObjectId	Unique payment identifier
userId	ObjectId	Reference to user
courseId	ObjectId	Purchased course reference
paymentId	String	Payment gateway transaction ID
Amount	Number	Paid amount
paymentStatus	String	Payment transaction status

5. WORK DONE

5.1 ***Development Environment***

The development of the Learning and Placement Preparation Platform was carried out using a modern full-stack web development environment designed to support scalability, modularity, responsiveness, and efficient backend processing. Since the project integrates multiple functionalities such as aptitude quizzes, coding preparation systems, ATS resume analysis, online learning modules, analytics dashboards, and authentication workflows, the development environment needed to support both frontend and backend operations efficiently.

The application was developed using the MERN stack architecture, which consists of MongoDB, Express.js, React.js, and Node.js. This technology stack was selected because it supports modern web application development, reusable frontend components, scalable backend APIs, and efficient database management using NoSQL architecture.

The frontend development environment was implemented using React.js along with Tailwind CSS. React.js was selected because of its component-based architecture, reusable UI design, efficient rendering system, and ability to support dynamic single-page applications. Tailwind CSS was integrated to design responsive and visually organized interfaces while reducing custom CSS complexity.

The backend environment was developed using Node.js and Express.js. Node.js was selected because of its asynchronous event-driven architecture and efficient handling of concurrent API requests. Express.js simplified REST API development and middleware integration, allowing the backend to manage authentication systems, ATS analysis workflows, analytics processing, and quiz management efficiently.

MongoDB Atlas was used as the cloud database management system because it supports flexible document-oriented storage and scalable data handling. Mongoose was integrated within the backend to implement schema validation and structured database management.

The complete development environment also included GitHub for version control and source code management. GitHub repositories were used to maintain frontend and backend source code separately while supporting incremental development workflows and deployment integration.

The frontend application was deployed using Vercel, while backend APIs were hosted using Render. These cloud deployment platforms were selected because of their scalability, automated deployment support, and compatibility with MERN stack applications.

The development environment also included Postman for API testing and debugging. Postman was extensively used during backend development to test authentication APIs, quiz management routes, ATS analysis endpoints, and course management functionalities.

Visual Studio Code was used as the primary code editor because it supports multiple extensions, integrated terminal functionality, debugging tools, and efficient JavaScript development workflows.

5.2 Software and Tools Used

Several modern development technologies and software tools were used during the implementation of the Learning and Placement Preparation Platform. These tools were selected carefully to support scalable full-stack web development, responsive frontend design, secure backend processing, cloud deployment, and efficient database management.

React.js: React.js was used for frontend development because of its component-based architecture and efficient rendering system. The framework supports reusable UI components, dynamic page rendering, and smooth frontend-backend communication. React.js also improved maintainability and simplified the implementation of dashboards, quiz interfaces, ATS analysis panels, analytics graphs, and course learning pages.

Tailwind CSS: Tailwind CSS was integrated for responsive user interface design and layout management. The utility-first CSS framework simplified frontend styling while improving consistency across different pages of the application. Tailwind CSS also helped implement responsive layouts compatible with desktop systems, tablets, and mobile devices.

Node.js: Node.js was used as the backend runtime environment because of its asynchronous and event-driven architecture. It efficiently handles concurrent API requests and supports scalable backend development. Node.js also improved frontend-backend communication and supported cloud deployment workflows.

Express.js: Express.js was used for backend API development and middleware management. The framework simplified the implementation of REST APIs responsible for authentication, quiz management, ATS analysis, analytics generation, and course management functionalities. Express.js also improved backend organization through modular routing and controller architecture.

MongoDB Atlas: MongoDB Atlas was used as the cloud database management system because of its scalability, flexible document-oriented storage, and cloud accessibility. MongoDB efficiently handled dynamic educational data such as user records, quiz questions, ATS reports, coding submissions, and course enrollments.

GitHub: GitHub was used for version control and source code management throughout the development lifecycle. Separate repositories were maintained for frontend and backend codebases. GitHub also supported incremental development workflows and deployment integration.

Visual Studio Code: Visual Studio Code was used as the primary development environment because of its lightweight architecture, integrated terminal support, debugging tools, and extension ecosystem. It simplified JavaScript development and improved overall coding efficiency.

Postman: Postman was used extensively for backend API testing and debugging. Authentication APIs, quiz routes, ATS analysis endpoints, and course management APIs were validated using Postman before frontend integration.

Vercel: Vercel was used for frontend deployment because it supports automated deployment pipelines and efficient hosting for React.js applications. It simplified frontend hosting and improved online accessibility.

Render: Render was used for backend deployment and API hosting. The platform supports scalable Node.js application deployment and simplifies backend server management.

5.3 Implementation and Module Development

The implementation phase focused on converting system design and functional requirements into a fully operational web application. The platform was developed incrementally using modular development methodology where each module was implemented independently before integration into the complete system.

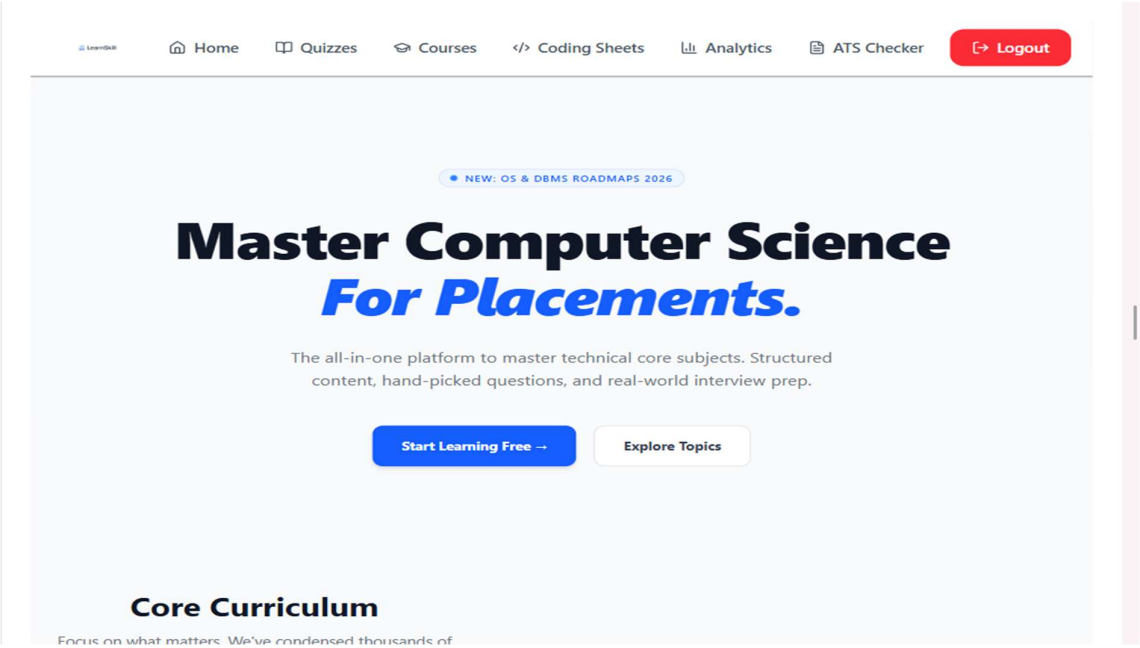


Figure 5.1 Home Page of the website

The implementation process began with backend API development and database schema creation. Authentication workflows were implemented first because secure user access was required before integrating protected functionalities such as quizzes, ATS analysis, and analytics dashboards.

JWT-based authentication mechanisms were integrated to manage secure sessions and protected route access. Password hashing techniques were implemented using bcrypt before storing user credentials inside MongoDB collections. Middleware validation systems were also integrated to verify authentication tokens during API communication.

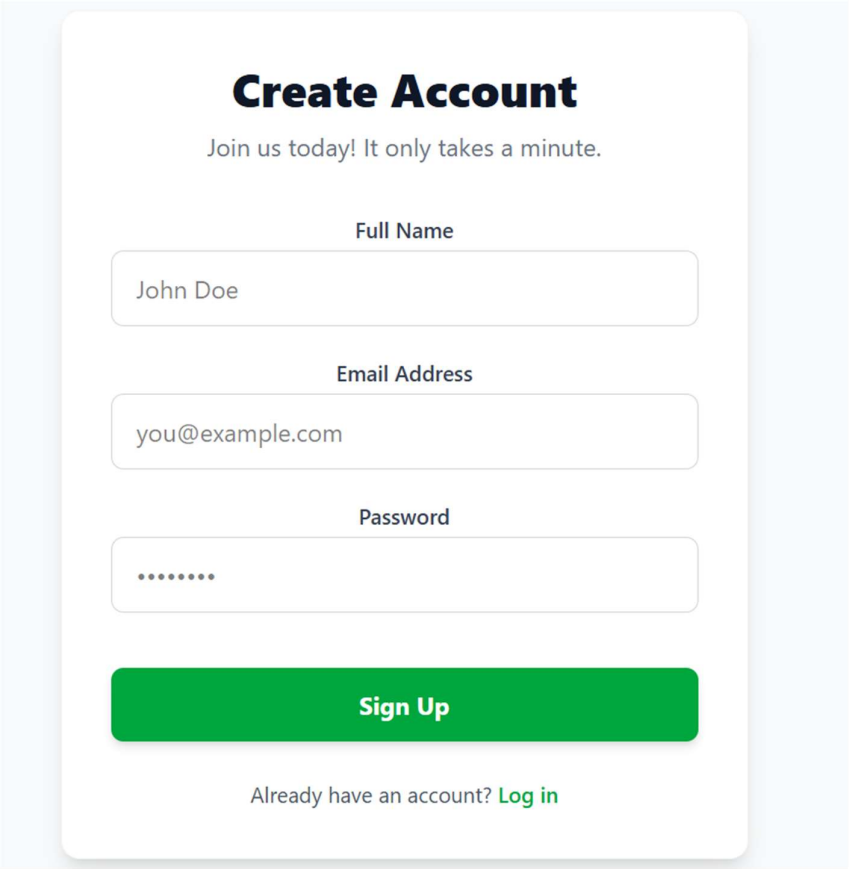


Figure 5.2 Registration Page

After implementing authentication workflows, the quiz management system was developed. The quiz module included backend APIs for question retrieval, quiz submission handling, score calculation, and result storage. Questions were categorized according to technical subjects such as DBMS, Operating Systems, Aptitude, and Computer Networks.

Frontend quiz dashboards were then implemented using React.js components. Dynamic quiz interfaces were created where users could select quiz categories, attempt questions, and view generated results immediately after submission.

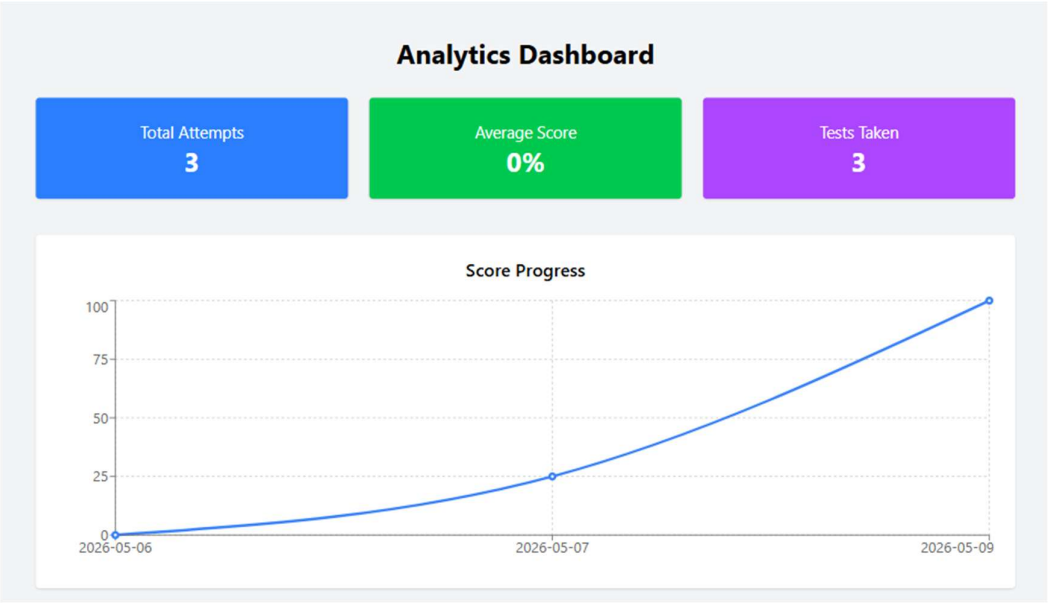
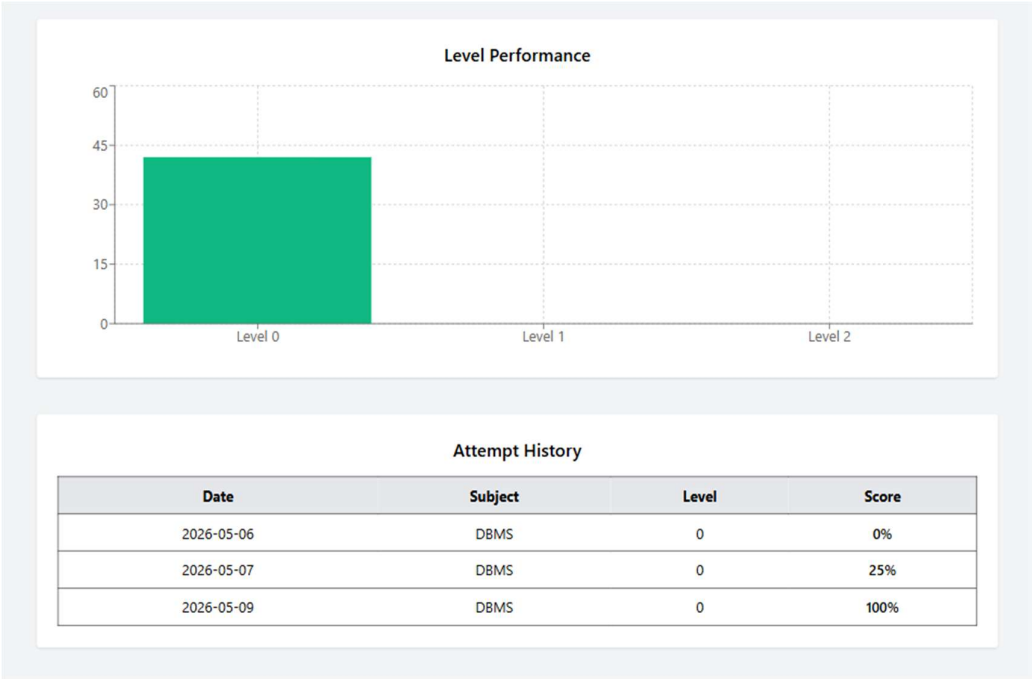


Figure 5.3: Analytics Page



Date	Subject	Level	Score
2026-05-06	DBMS	0	0%
2026-05-07	DBMS	0	25%
2026-05-09	DBMS	0	100%

Figure 5.4 Analytics Page

The coding preparation module was developed next. Coding questions were categorized according to topics and difficulty levels. Separate MongoDB collections were created to store coding problems and coding submissions. Backend APIs were implemented to retrieve coding questions dynamically while frontend dashboards allowed users to practice structured technical interview problems.

One of the most important modules implemented during development was the ATS Resume Analysis system. This module integrates resume optimization functionalities into the platform.

The ATS workflow begins when users upload resumes and provide job descriptions through frontend interfaces.

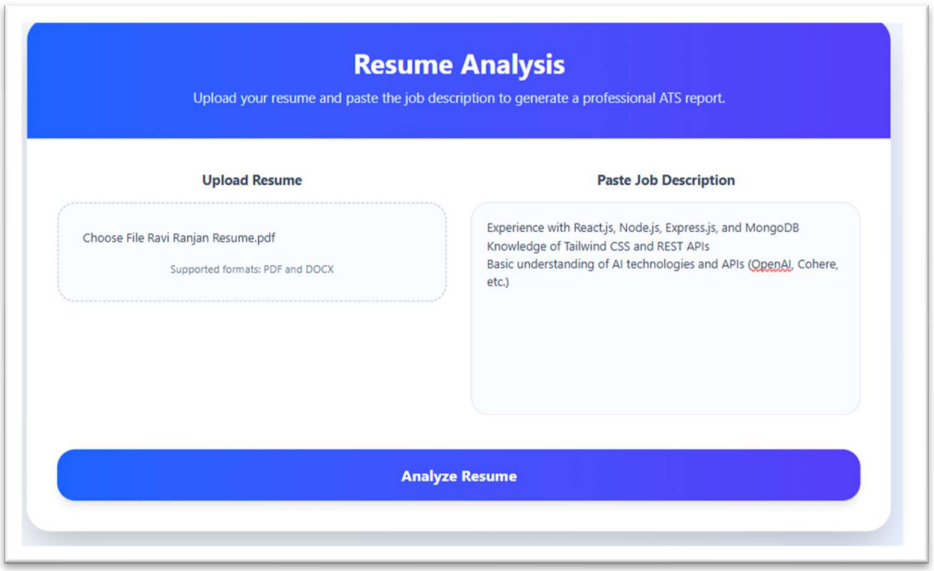


Figure 5.5

Figure 5.6 ATS Resume Analysis Interface

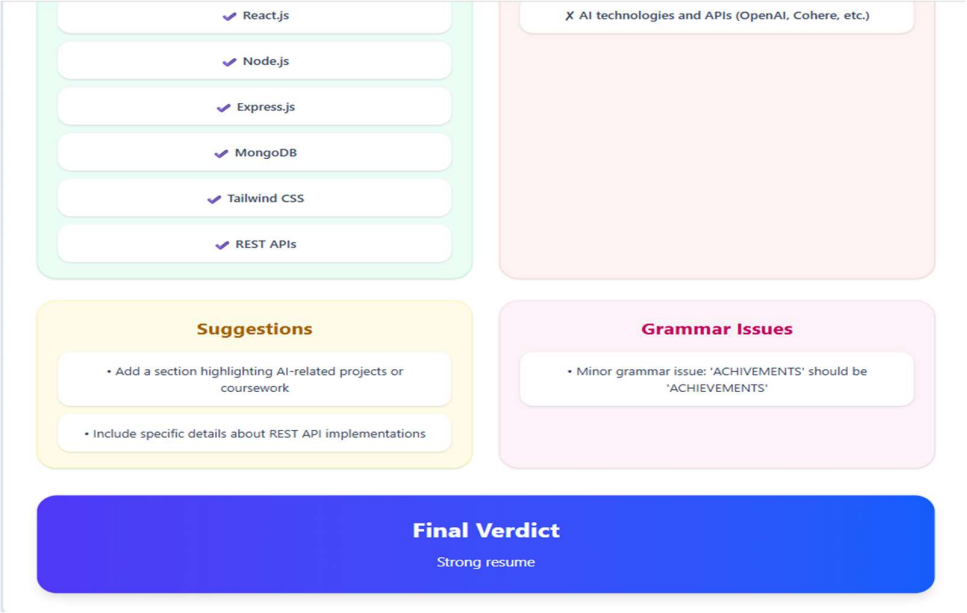


Figure 5.7 ATS Resume Analysis Interface

The backend extracts textual content from uploaded resumes and generates prompts dynamically for AI-based analysis systems. The generated ATS reports contain ATS scores, keyword matching analysis, missing technical skills, and resume improvement suggestions. These reports are stored inside MongoDB collections and displayed through frontend dashboards.

The implementation of the ATS system significantly improved the practical value of the platform because students preparing for placements can optimize resumes according to industry recruitment standards.

The course management and online learning system was implemented after the ATS module. Backend APIs were developed for course creation, module retrieval, and enrollment handling. React.js dashboards were created to display enrolled courses, course details, learning modules, and progress tracking systems.

The platform architecture was also designed to support future payment gateway integration for paid course enrollment functionalities. Course schemas include pricing-related fields, while the database architecture supports future transaction and payment management systems.

Analytics dashboards were implemented to visualize user performance and preparation progress. Historical quiz records and learning progress data were retrieved from MongoDB collections and displayed using graphical representations created with Recharts and Chart.js libraries.

The analytics dashboards display:

- Quiz score trends

- Historical performance graphs
- Progress tracking reports

This visualization system improves self-assessment and helps students monitor preparation consistency effectively.

Frontend routing was implemented using React Router to manage navigation between authentication pages, dashboards, quizzes, ATS systems, analytics panels, and course learning modules.

The application was tested continuously during implementation to ensure proper frontend-backend communication and database consistency. Postman was used extensively to test backend APIs while frontend responsiveness was verified across multiple devices and screen sizes.

5.4 ***Testing and Validation***

Testing and validation form one of the most significant phases in the software development lifecycle because they ensure that the developed application performs accurately, securely, and efficiently under practical operating conditions. The Learning and Placement Preparation Platform integrates several independent yet interconnected modules such as user authentication systems, aptitude and technical quiz modules, coding preparation dashboards, ATS-based resume analysis systems, analytics dashboards, and online learning functionalities. Since these modules continuously exchange information through frontend interfaces, backend APIs, MongoDB databases, and external NLP-based services, comprehensive testing was essential to guarantee stable communication, reliable performance, and production-level usability.

The testing phase was not limited to the final stage of development. Instead, testing activities were carried out continuously throughout implementation so that errors could be identified and resolved at early stages. This approach significantly reduced debugging complexity and improved overall software quality. Continuous testing also ensured that newly developed functionalities did not negatively affect previously implemented modules.

The complete testing process focused on validating application behavior under multiple user scenarios. Authentication workflows, database communication, ATS analysis systems, frontend responsiveness, analytics generation, cloud deployment stability, and overall user interaction were evaluated carefully. Different testing methodologies were adopted to examine the system from both functional and usability perspectives. These methodologies ensured that the platform remained reliable, responsive, scalable, secure, and suitable for deployment in a real-world educational environment.

5.4.1 Testing Objectives

The primary objective of testing was to ensure that the Learning and Placement Preparation Platform operated correctly and fulfilled all functional and non-functional requirements identified during requirement analysis and system design phases. Since the platform was developed as a centralized environment for placement preparation, coding practice, resume optimization, analytics monitoring, and online learning, it was important to validate that every module performed accurately under real-world usage conditions.

One of the major goals of testing was to verify smooth communication between frontend interfaces and backend APIs. The frontend developed using React.js continuously exchanges data with backend services developed using Node.js and Express.js. Therefore, validating stable request processing, proper response handling, and secure API communication was extremely important.

Another important testing objective was to evaluate authentication and authorization mechanisms. Since the platform stores sensitive information such as user credentials, ATS reports, quiz histories, coding progress, and learning records, it was necessary to ensure that unauthorized users could not access protected functionalities or manipulate stored data.

Testing also aimed to validate the correctness of quiz score calculation systems, ATS report generation workflows, analytics visualization modules, and course management functionalities. The platform needed to ensure that generated quiz results, ATS scores, graphical dashboards, and progress tracking systems produced accurate and consistent outputs.

Responsive frontend behavior was another important objective because students may access the platform from different devices such as desktops, laptops, tablets, and smartphones. Therefore, the testing process evaluated interface responsiveness and cross-browser compatibility carefully.

Additionally, deployment testing objectives included validating cloud hosting environments, MongoDB Atlas connectivity, CORS configuration, environment variable management, and production-level API accessibility. Overall, the testing objectives focused on ensuring application stability, usability, scalability, reliability, and security.

5.4.2 Testing Methodologies

Several software testing methodologies were adopted during the development of the platform to validate both independent modules and complete system integration. Since the project combines multiple functionalities into a unified web-based environment, different testing approaches were necessary to evaluate software quality comprehensively.

Unit testing was performed to verify the behavior of individual modules independently before integration into the complete application. Backend controllers, authentication systems, quiz

evaluation functions, ATS analysis workflows, and course management APIs were tested separately to identify implementation-level errors at early stages.

Integration testing was performed after unit testing to validate communication between frontend interfaces, backend APIs, MongoDB collections, middleware systems, and external ATS analysis services. This methodology ensured that independent components exchanged information correctly without causing failures or inconsistencies.

System testing focused on validating the complete platform after integrating all modules together. The entire application was tested under practical operating conditions to evaluate workflows such as user registration, login authentication, quiz participation, ATS analysis, analytics visualization, and online learning management.

Validation testing was also carried out to ensure that the implemented system satisfied all project objectives identified during requirement analysis. This methodology focused on verifying that the final application delivered the intended functionality and user experience.

Security testing methodologies were adopted to validate JWT-based authentication systems, protected API routes, password encryption mechanisms, and middleware authorization workflows. Unauthorized access attempts and invalid request scenarios were simulated to evaluate secure access control.

Responsive testing and UI testing methodologies were applied to evaluate frontend rendering behavior across multiple screen sizes, browsers, and operating conditions. This ensured that the platform remained visually consistent and accessible across all supported devices.

The use of multiple testing methodologies significantly improved application quality, maintainability, scalability, reliability, and production readiness.

5.4.3 ***Tools Used for Testing***

A variety of software tools and development utilities were used during testing and validation to simplify debugging, API verification, frontend inspection, deployment monitoring, and database management. These tools played an important role in improving software stability and reducing development complexity.

Postman was one of the primary tools used during backend API testing. It was extensively used to test authentication APIs, quiz retrieval systems, ATS analysis endpoints, course management routes, and analytics-related APIs. Postman allowed backend requests to be executed independently before frontend integration, making it easier to identify request handling errors, response inconsistencies, and middleware failures.

MongoDB Atlas and MongoDB Compass were used for database validation and collection inspection. These tools helped verify correct insertion, retrieval, updating, and deletion of documents related to users, quizzes, ATS reports, coding questions, enrollments, and analytics records. Database inspection tools also simplified debugging of ObjectId references and schema relationships.

Browser Developer Tools available in Google Chrome and Microsoft Edge were used extensively during frontend testing and responsive validation. These tools helped inspect frontend rendering behavior, monitor API requests, debug CSS issues, and evaluate responsive layouts across different screen resolutions.

Visual Studio Code debugging utilities were also used throughout development and testing. Console logging, breakpoint debugging, and terminal inspection features simplified identification of backend logic errors, middleware failures, and frontend component rendering issues.

Chart.js and Recharts outputs were validated using browser-based visualization testing to ensure proper graphical rendering and responsive analytics behavior.

Deployment testing involved cloud hosting environments such as Vercel, Render, and MongoDB Atlas. Deployment logs, runtime monitoring systems, and browser-based production testing were used to validate application accessibility and API communication stability after deployment.

These testing tools collectively improved API reliability, frontend responsiveness, database consistency, debugging efficiency, and production-level stability of the platform.

5.4.4 Types of Testing Performed

Manual and UI Testing

Manual testing was performed extensively because the platform contains several interactive workflows that require direct user interaction. Unlike automated testing, manual testing helped evaluate practical usability, frontend navigation behavior, ATS report visualization quality, and overall user experience from a real-user perspective.

Authentication interfaces were tested manually by entering different combinations of valid and invalid user inputs. Registration forms were evaluated using incorrect email formats, weak passwords, duplicate accounts, and incomplete submissions to ensure proper validation handling. Login systems were also tested using valid credentials, incorrect passwords, and unauthorized access attempts.

Quiz interfaces required significant manual testing because they represent one of the primary functionalities of the platform. Users were able to select subjects, navigate through questions, select

answers, and submit quizzes smoothly. Result generation workflows and historical performance records were also tested manually to ensure correct frontend visualization.

ATS Resume Analysis workflows were tested carefully because they involve multiple user interactions including resume uploads, job description submission, and ATS report interpretation. Different resumes and job descriptions were uploaded repeatedly to evaluate report consistency, frontend rendering quality, and response formatting.

Manual testing was also carried out for coding preparation modules, analytics dashboards, and course management systems. Users were able to browse learning modules, access lessons, track progress, and monitor analytics without interface failures.

UI testing focused on validating frontend responsiveness, layout consistency, chart rendering behavior, typography alignment, navigation smoothness, and component positioning. Since the frontend was developed using Tailwind CSS, responsive rendering across multiple devices was extremely important.

Responsive testing was performed on desktop systems, laptops, tablets, and smartphones. Tailwind CSS layouts adapted correctly to different screen sizes without affecting visual structure or user interaction quality.

Cross-browser testing was also performed on Google Chrome, Mozilla Firefox, and Microsoft Edge to ensure frontend compatibility and consistent rendering behavior across different browsers.

Manual and UI testing significantly improved platform usability, responsive rendering quality, interface consistency, and overall user experience.

Unit Testing

Unit testing focused on validating individual modules and functionalities independently before complete system integration. Each backend controller, middleware function, authentication workflow, database operation, and ATS processing system was tested separately to identify errors within isolated modules.

Authentication systems were tested independently to validate user registration workflows, password hashing operations, login verification systems, and JWT token generation mechanisms. Quiz score calculation systems were also tested separately using different combinations of correct and incorrect answers to verify accurate score generation and percentage calculations.

ATS Resume Analysis modules required independent testing because they involve file handling, text extraction, prompt generation, and API communication workflows. Resume upload systems and ATS report generation functions were validated separately before frontend integration.

Coding preparation APIs and course management systems were also tested independently to ensure correct retrieval of coding questions, course information, and progress tracking records.

Unit testing reduced debugging complexity significantly because errors could be isolated and corrected before affecting other components of the application.

Integration Testing

Integration testing was performed after successful unit testing to validate communication between frontend interfaces, backend APIs, MongoDB collections, middleware systems, and external ATS analysis services.

Frontend React.js components successfully communicated with backend Express.js APIs through HTTP requests. Quiz interfaces retrieved questions dynamically from MongoDB collections while backend score processing systems generated accurate results successfully.

ATS Resume Analysis workflows involving resume uploads, backend processing systems, Cohere API communication, and frontend report visualization were tested carefully to ensure smooth integration between all involved components.

Authentication systems were integrated successfully with protected API routes and frontend dashboard access control mechanisms. JWT tokens generated during login were validated correctly during future API requests.

Course management systems, analytics dashboards, coding preparation modules, and MongoDB database operations were also validated successfully during integration testing.

Integration testing confirmed that all independent modules communicated correctly and functioned together as a unified full-stack web application.

System Testing

System testing focused on validating the complete platform as an integrated production-level application. Real-world workflows such as account registration, login authentication, quiz participation, ATS analysis, analytics monitoring, coding preparation, and online learning activities were tested sequentially.

The complete platform operated successfully without frontend crashes, backend failures, or database inconsistencies. Navigation between different modules remained smooth and responsive throughout testing.

System testing also evaluated application behavior under continuous API requests and simultaneous user operations. Backend APIs processed requests reliably while MongoDB collections maintained consistent data storage behavior.

Responsive behavior across multiple devices and browsers was also validated during system testing to ensure consistent user experience under practical usage conditions.

System testing confirmed that the complete application functioned successfully as a centralized placement preparation and learning management platform.

5.5 Results and Discussion

The final implementation of the Learning and Placement Preparation Platform successfully achieved the major objectives identified during requirement analysis and system design phases. The developed application integrates placement preparation functionalities, coding practice systems, ATS resume analysis, learning management modules, analytics dashboards, and authentication workflows into a centralized web-based environment.

The frontend application was successfully deployed on Vercel while backend APIs were deployed using Render. MongoDB Atlas cloud hosting ensured efficient database accessibility and scalability throughout application execution.

The authentication system operated successfully using JWT-based session management and middleware authorization workflows. Users were able to register securely, log into personalized dashboards, and access protected functionalities without security issues.

The quiz management system successfully supported dynamic question retrieval, quiz participation, score calculation, and historical performance tracking. Quiz analytics helped users monitor preparation consistency and identify weak technical subjects requiring improvement.

The coding preparation system also functioned successfully by organizing coding problems according to topics and difficulty levels. This structure improved technical interview preparation workflows and supported systematic learning.

The ATS Resume Analysis module produced effective ATS evaluation reports by comparing resumes against job descriptions. The system generated ATS scores, missing keyword analysis, and improvement recommendations successfully. This module significantly improved the practical usefulness of the platform because resume optimization is an important part of placement preparation processes.

The online learning and course management system also operated successfully. Users were able to browse learning modules, access course content, and monitor learning progress through dynamic dashboards.

The analytics system effectively visualized quiz histories and learning progress using graphical dashboards. Performance visualization through graphs and charts improved user understanding and preparation tracking.

The modular architecture of the application significantly improved maintainability and scalability. Backend APIs remained organized through MVC architecture while frontend interfaces remained reusable and responsive through React.js component-based development.

Cloud deployment workflows using Vercel, Render, and MongoDB Atlas ensured online accessibility and production readiness of the application.

The project also demonstrated the effectiveness of integrating multiple placement preparation functionalities into a single platform. Instead of depending on separate websites for quizzes, coding practice, resume analysis, and learning management, users can access all functionalities through one centralized system.

The implementation process also highlighted several practical advantages of MERN stack architecture including reusable frontend development, scalable REST APIs, flexible MongoDB schema design, and cloud-native deployment support.

Although the platform successfully implemented major functionalities, future enhancements can further improve system capabilities. Features such as online coding compilers, payment gateway integration, AI-based recommendation systems, mock interview simulators, certificate generation systems, and mobile application support can be integrated in future versions.

Overall, the developed platform successfully fulfilled the primary objective of providing students with a centralized environment for placement preparation, technical learning, resume optimization, and performance monitoring.

6. CONCLUSION AND FUTURE SCOPE

6.1 ***Conclusions***

This project set out to build a unified platform that addresses the fragmented nature of placement preparation resources available to engineering students. Over the course of development, a complete full-stack web application was designed and implemented using MongoDB, Express.js, React.js, and Node.js, and was successfully deployed to cloud infrastructure using Render for the backend services and Vercel for the frontend application.

The platform successfully integrates four major preparation modules within a single system: a CS Fundamentals Quiz System with analytics and performance tracking, a Coding Preparation Module with topic-wise and difficulty-based filtering, an ATS Resume Checker integrated with the Cohere API, and a Paid Course Learning Platform with Razorpay payment integration and real-time course progress tracking. All modules operate through a shared authentication system based on JSON Web Tokens (JWT), a consistent Tailwind CSS-based user interface, and a centralised MongoDB database for storing user data, progress history, quiz results, and course information.

From a technical perspective, the project involved solving several real-world development challenges. The integration of the Cohere API for resume analysis required careful prompt structuring to ensure that the generated responses remained consistent, readable, and suitable for frontend display. Additionally, the quiz analytics system demonstrated the effectiveness of MongoDB aggregation pipelines for processing and visualising performance data in real time.

The development process also provided extensive practical experience in the complete software development lifecycle, including database schema design, RESTful API development, frontend component architecture, authentication handling, deployment workflows, and production environment configuration. Multiple technical challenges encountered during implementation — such as CORS handling, backend cold-start latency on Render, PDF resume text extraction issues, and variations in API response formatting — were resolved through iterative debugging, testing, and optimisation.

In conclusion, the Learning and Placement Preparation Platform successfully achieves the objectives defined at the beginning of the project. It demonstrates the practical application of modern full-stack web development technologies in solving a real educational problem while delivering a functional and user-focused solution for engineering students preparing for placements. The platform is fully deployed, publicly accessible, and capable of supporting integrated learning, practice, analytics, and resume evaluation within a single environment.

6.2 Future Scope of Work

Learning and Placement Preparation Platform has been developed as a centralized solution for placement preparation, coding practice, AI-based resume analysis, and online learning management. Although the current system successfully achieves its major objectives, there is significant scope for future enhancements and scalability. The platform architecture has been designed in a modular manner, making it suitable for integrating advanced technologies and additional functionalities in future versions.

6.2.1 AI-Based Personalized Learning Recommendations

Future versions of the platform can include machine learning algorithms to provide personalized learning recommendations based on user performance and activity history. The system can analyze quiz scores, coding performance, and learning progress to recommend suitable coding problems, quizzes, and courses. This would help users focus on weak areas and improve preparation efficiency.

6.2.2 AI Mock Interview System

An AI-powered interview preparation module can be added to simulate technical and HR interviews. The system can ask interview questions, analyze user responses, and provide performance feedback. Natural Language Processing and speech recognition technologies can further enhance the interview simulation experience and help students improve communication and problem-solving skills.

6.2.3 Mobile Application Development

The platform can be expanded into a mobile application using technologies such as React Native or Flutter. A mobile application would allow students to access quizzes, coding preparation, ATS analysis, and courses directly from smartphones, improving accessibility and user engagement.

REFERENCES

[1] React.js Official Documentation, “React Documentation,” Available: https://react.dev

[2] Node.js Foundation, “Node.js Documentation,” Available: https://nodejs.org/en/docs

[3] Express.js, “Express Web Framework Documentation,” Available: https://expressjs.com

[4] MongoDB Inc., “MongoDB Documentation,” Available: https://www.mongodb.com/docs

[5] Mongoose, “Mongoose ODM Documentation,” Available: https://mongoosejs.com/docs

[6] Tailwind Labs, “Tailwind CSS Documentation,” Available: https://tailwindcss.com/docs

[7] Auth0, “JSON Web Token Introduction,” Available: https://jwt.io

[8] Cohere AI, “Cohere API Documentation,” Available: https://docs.cohere.com

[9] Chart.js Contributors, “Chart.js Documentation,” Available: https://www.chartjs.org/docs

[10] Recharts Group, “Recharts Documentation,” Available: https://recharts.org/en-US

[11] Razorpay Software Pvt. Ltd., “Razorpay Documentation,” Available: https://razorpay.com/docs

[12] Render Services Inc., “Render Documentation,” Available: https://render.com/docs

[13] Vercel Inc., “Vercel Documentation,” Available: https://vercel.com/docs

[14] GitHub Inc., “GitHub Documentation,” Available: https://docs.github.com

[15] Postman Inc., “Postman API Platform Documentation,” Available: https://learning.postman.com/docs

[16] Mozilla Developer Network, “JavaScript Documentation,” Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[17] HTML Living Standard, “HTML Documentation,” Available: https://html.spec.whatwg.org